

Reserved Quantity Akışı — Takas · Teklif · Ödeme

Tarih: 2026-06-24 · Kapsam: [apps/api](#) — [Product.reservedQuantity](#) ne zaman artar/azalır. Amaç: takas (trade), teklif (offer) ve ödeme (payment) yaşam döngülerinde rezerve adedin **mevcut** davranışını birebir koddan haritalamak. Her satır koddan teyit edilmiştir; bu doküman kodu değiştirmez, **şu anki durumu** anlatır.

İlgili denetim notları (hatalar + öneriler) için: [stok-quantity-denetim.md](#).

1. Model

Dosya: [product-availability.helper.ts](#)

Alan	Anlamı
<code>quantity</code>	Fiziksel stok. <code>null</code> = sınırsız. Yalnız ödeme başarısında / takas tamamlanmasında düşer.
<code>reservedQuantity</code>	Tutulan (ödeme bekleyen / takası süren) adet.
<code>available</code>	<code>getAvailableQuantity = max(0, quantity - reservedQuantity)</code> ; <code>quantity=null</code> ise <code>null</code> (sınırsız).
<code>status</code>	<code>draft · pending · active · reserved · sold · inactive · rejected · deleted</code>

Yardımcılar:

- `getAvailableQuantity(product)` — [satır 8-17](#)
- `safeDecrementReserved(current, by) = max(0, (current ?? 0) - by)` — asla negatife düşmez. [satır 23-28](#)
- `checkAndReserve(tx, productId, qty)` — rezerve almanın **tek kapısı**. [product-lock.service.ts:106-140](#):
 - Ürünü `FOR UPDATE` ile pessimistic kilitler.
 - `status` `active` veya `reserved` değilse hata fırlatır.
 - `available < requiredQty` ise "stokta yok" hatası fırlatır.
 - `reservedQuantity += requiredQty` (**status'a DOKUNMAZ**).

⚠ Kritik nokta: `checkAndReserve` rezerve adedi artırır ama ürün `status` 'unu `reserved` yapmaz. Status'u ayrıca çağırان taraf set eder (takas kabulü yapar; satın alma yolu yapmaz — bkz. denetim notu A).

2. Temel Kural (özet)

- Rezerve ne zaman ALINIR?**
 - Takas:** kabul anında (cash + non-cash ortak).
 - Direct-buy (hemen al):** sipariş oluşturulurken.
 - Teklif (offer):** teklif kabulünde DEĞİL → **ödeme başlatıldığında** (payment initiate).
- quantity ne zaman DÜŞER?** Yalnız ödeme **başarısında** veya takas **tamamlanmasında**. Sipariş/teklif/takas oluşturmada asla.
- Rezerve ne zaman BIRAKILIR?** Ödeme başarısında (quantity'ye dönüşerek), ödeme fail/iptal/süre-aşımında, takas red/iptal/expire'inde.

3. Takas (Trade) Yaşam Döngüsü

Dosya: [trade.service.ts](#)

#	Olay	Tetikleyici	reserved	Δ	Status değişir mi?	Dosya:satır
1	Teklif oluştur (pending)	<code>POST /trades</code>	—	yok	hayır	— (yorum: kabule kadar rezerve etme)
2	Takas kabul	<code>acceptTrade</code>	artar	<code>+qty</code> her iki taraf	<code>available<=0</code> ise $\rightarrow$ <code>reserved</code>	908 / 920
3	Cash ödeme başarısı	PayTR callback	değişmez	yok (zaten kabulde alındı)	takas $\rightarrow$ <code>shipping_to_warehouse</code>	—
4	Reddet (accepted ise)	<code>rejectTrade</code>	azalır	<code>safeDecrementReserved(_, qty)</code>	$\rightarrow$ <code>active</code> (koşulsuz)	1066-1069
5	İptal (non-pending ise)	<code>cancelTrade</code>	azalır	<code>safeDecrementReserved(_, qty)</code>	$\rightarrow$ <code>active</code> (koşulsuz)	1422-1425
6	Tamamlanma (alıcı onayı)	<code>confirmReceipt</code>	azalır	<code>quantity-- + reserved--</code>	<code>getProductStatusFromQuantity</code>	1714
6b	Tamamlanma (oto-onay)	<code>autoConfirmExpiredReceipts</code>	azalır	<code>quantity-- + reserved--</code>	<code>getProductStatusFromQuantity</code>	1923 / 2250
7	Süre-aşımı iptal	<code>autoCancelExpiredTrades</code>	azalır	<code>safeDecrementReserved(_, qty)</code>	$\rightarrow$ <code>active</code>	1944 / 2083-2087

Detaylar

(2) **Kabul** — `acceptTrade` : [trade.service.ts:906-908](#)

```
// Takas kabul: her iki taraf için reservedQuantity++ (FOR UPDATE pessimistic lock)
for (const [productId, qty] of byProduct) {
  await this.productLockService.checkAndReserve(tx, productId, qty);
}
```

Rezerve **tam 1 kez** burada alınır — hem teklif edenin hem alıcının ürünleri için. Sonrasında (920) `available = quantity - reservedQuantity` yeniden okunur; `<=0` ise ürün `status=reserved` yapılır ve diğer bekleyen teklif/takas/sipariş cascade iptal edilir.

- Cash takas $\rightarrow$ `awaiting_payment` (nakit ödenmeden sevkiyat yok).
- Non-cash takas $\rightarrow$ `shipping_to_warehouse`.

(3) **Cash ödeme başarısı**: Rezerve zaten kabulde alındığı için **tekrar rezerve ETMEZ** — sadece takası `shipping_to_warehouse` 'a taşır.

(6) **Tamamlanma**: [trade.service.ts:1689-1714](#) — her iki tarafın tüm `tradeItem` 'ları için `quantity--` ve `reserved--` aynı transaction'da. `quantity` ilk kez burada düşer.

4. Teklif (Offer) + Ödeme Yaşam Döngüsü

Dosyalar: [offer.service.ts](#), [payment.service.ts](#), [order.service.ts](#)

#	Olay	Tetikleyici	reserved	Δ	Dosya:satır
1	Teklif oluştur (pending)	POST /offers	—	yok	—
2	Teklif kabul → order(pending_payment)	POST /offers/:id/accept	—	yok (kasıtlı)	— (yorum: reserve ödeme başlatınca)
3	Ödeme başlat (offer order)	initiatePayment	artar	+1	604
3b	Ödeme tekrar başlat (30dk bırakma sonrası)	initiatePayment retry	artar	+1 (CAS-gate'li)	616
4	Ödeme başarılı	PayTR callback	azalır	quantity-- + reserved--	1369
5	Ödeme fail / iptal	releaseProductForFailedPayment	azalır	safeDecrementReserved(____, 1)	1953-1954

Detaylar

(2) Kabul — reserve YOK: Teklif kabulü yalnız "anlaşma"dır. Stok değişmez, invalidation yapılmaz. Bunun yerine `pending_payment` order üretilir; reserve ödeme başlatılınca alınır.

(3) Ödeme başlat — `initiatePayment` : `payment.service.ts:601-606`

```
if (order.offerId && !order.reservationReleasedAt) {
  await this.productLockService.checkAndReserve(tx, order.productId, 1);
}
```

- Guard: `order.offerId` var **ve** reserve daha önce bırakılmamış (`!reservationReleasedAt`).
- (3b)** 30dk cron rezervayı bıraktıysa (`reservationReleasedAt` set), retry yolu **616** CAS-gate ile tek seferlik yeniden reserve eder.
- Karşılaştırma: direct-buy `offerId=null` olduğu için bu blok ona dokunmaz (o sipariş oluşumunda reserve edilmiştir).

(4) Ödeme başarılı: `payment.service.ts:1355-1372` (group için: `1729-1730`)

```
const newQuantity = product.quantity !== null ? product.quantity - 1 : null;
updateData.status = getProductStatusFromQuantity(newQuantity);
updateData.reservedQuantity = safeDecrementReserved(product.reservedQuantity, 1);
if (product.quantity !== null) updateData.quantity = { decrement: 1 };
```

Rezerve → fiziksel satışa dönüşür: `quantity--` **ve** `reserved--` aynı anda. Sonra `available<=0` ise stockout cascade (`1388-1421`) bekleyen diğer order/offer'ları iptal eder.

Üyelik/boost siparişleri stoğa dokunmaz; bu blok yalnız regular ürün siparişlerinde çalışır.

(5) Ödeme fail/iptal — `releaseProductForFailedPayment` : `payment.service.ts:1951-1958`

```
const newReserved = safeDecrementReserved(before.reservedQuantity, 1);
updateData.reservedQuantity = newReserved;
if (before.status === ProductStatus.reserved && newReserved === 0) {
  updateData.status = ProductStatus.active;
}
```

Offer-kökenli order fail olursa teklif `payment_expired` yapılır → alıcı tekrar ödeyebilir (yeniden kabule gerek yok).

5. Direct-Buy / Grup Checkout (karşılaştırma)

Dosya: `order.service.ts`

Olay	reserved	Δ	Dosya:satır
Direct-buy sipariş oluştur	artar	+1 (oluşumda)	order.service.ts (~`reservedQuantity: { increment: 1 }`)
Grup checkout (per order)	artar	+1 her order	order.service.ts (~grup reserve döngüsü)
Ödeme başarısı	azalır	quantity-- + reserved--	payment.service.ts:1369 / group 1730

Teklif yolundan **tek fark**: reserve oluşumda alınır (offer'da ödeme başlatınca alınır). Ödeme başarısı/fail davranışı aynıdır.

6. Cron / Scheduler Etkileri

Dosya: [payment-scheduler.service.ts](#), [payment.service.ts](#)

Cron	Aralık	reserved etkisi	Dosya:satır
<code>releaseExpiredOrderReservations</code>	5 dk	30dk boşta order → <code>reserved--</code> , <code>reservationReleasedAt</code> set	payment.service.ts:3895-3900
<code>expireUnpaidOrders</code>	—	süresi dolan order → <code>reserved--</code> (<code>!reservationReleasedAt</code> guard'lı)	payment.service.ts:4047-4055
<code>reconcileReservedQuantities</code>	—	<code>reservedQuantity = held</code> (canlı pending_payment order sayımı) — ground-truth	payment.service.ts:3943-3989
<code>sweepOutOfStockProducts</code>	5 dk	<code>quantity=0</code> ürünlerde bekleyen offer/trade iptal	payment-scheduler.service.ts
<code>autoCancelExpiredTrades</code>	—	süresi dolan takas → <code>reserved--</code>	trade.service.ts:2083

7. Değişmezler (Invariants) — şu anki gar-antiler

- Asla negatif değil**: tüm decrement'ler `safeDecrementReserved` / SQL `GREATEST(...,0)` ile clamp'li.
- Rezerve tek kez alınır**: takas kabulünde / direct-buy oluşumunda / offer ödeme başlatmada. Cash ödeme başarısı tekrar rezerve etmez.
- quantity tek kez ve geç düşer**: yalnız ödeme başarısı + takas tamamlanması; her ikisinde de aynı transaction'da `reserved--` ile birlikte.
- Stockout cascade**: `available<=0` olunca ürün `reserved` 'a alınır + bekleyen diğer offer/trade/order iptal edilir (oversell önlemi).
- Reconcile son savunma**: canlı pending_payment order'ları sayıp `reservedQuantity` 'yi ground-truth'a çeker.

8. Bilinen açık noktalar (kısa)

Tam liste [stok-quantity-denetim.md](#)'de. Reserved akışını doğrudan etkileyenler:

- A** — `checkAndReserve` status'a dokunmaz; satın alma yolu `available<=0` 'da `reserved` yapmıyor → rezerve ürün bazı listelerde "stokta" görünebilir.
- B** — Sipariş iptalinde `reservationReleasedAt` guard + clamp eksik → çift bırakma → reserved negatif riski.
- D** — `reconcileReservedQuantities` ödemesi başlamamış offer order'ları da `held` sayıyor → reserved şişip "stokta yok" gösterebilir.